

直方图均衡化：是通过拉伸像素强度分布范围来增强图像对比度的一种方法。均衡化指的是把一个分布(给定的直方图)映射到另一个分布(一个更宽统一强度值分布)，所以强度值分布会在整个范围内展开。要想实现均衡化的效果映射函数应该是一个累积分布函数(cdf)。对于直方图H(q)，它的累积分布H'(q)= $\sum_{j=0}^q H(q_j)$ 。要使用其作为映射函数我们必须对最大值为255(或者用图像的最大强度值)的累积分布H'(q)进行归一化，最后使用一个简单的映射过程来获得均衡化后像素的强度值。代码：`gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY); equ = cv2.equalizeHist(gray)`。均衡化前后对比：原直方图集中在某些强度值上 → 均衡化后直方图均匀分布 → 图像视觉效果更清晰。**均衡化 ≠ 阈值处理!**

图像阈值处理是一种分离图像的的方法。通过阈值处理图像像素值取单一，图像内容减少。ret, dst = cv2.threshold(src, thresh, maxval, type)。thresh: 阈值，对于 8bit 灰阶图像取值范围为 0 至 255。maxval: 像素最大值行为取决于 type。type: 阈值类型一共 5 种 cv2.THRESH_BINARY(黑白二值)、cv2.THRESH_BINARY_INV(黑白二值反色) 等。

空间域图像域。直接对像素值处理 (如像素级叠加)。用灰度值描述图像。**频率域**表征灰度变化剧烈程度，是灰度在平面上的梯度。任何波形可分解为多个正弦波和，各含幅度和幅频。对图像做二维 DFT 将空间域转换到频率域。

图像中的**高频分量**指亮度/灰度变化剧烈处，即**边缘(轮廓)**；**低频分量**指亮度/灰度变换平缓处。低频量整幅图像强度综合，高频度量边缘和轮廓。

平滑滤波是低频增强**的空间域滤波技术**，目的为模糊或消除噪声。采用简单平均法求邻近元元平均亮度值，领域越大平滑效果越好但边缘信息损失越大，需合理选择领域大小。

处理要求：(1) 不损坏轮廓及边缘；(2) 使图像清晰。滤波函数需核模板，将模板放在像素 A 上求对应像素点之和。

OpenCV 函数 `cv.filter2D()` 对图像进行卷积。以 5×5 平均核为例：将核放在像素 A 上，求对应 25 个像素和再取平均代替 A 的值，重复直至所有像素更新完毕。

高斯滤波：卷积核换成高斯核，中心值最大、边缘递减构成高斯分布，求加权平均。`cv2.GaussianBlur(src, ksize, sigmaX)`——核宽高须为奇数，指定 X/Y 方向标准差。

中值滤波：`cv2.medianBlur(src, ksize)`——用卷积框中像素的**中值代替中心值**来达到去噪声的目的，这个模板一般用于去除椒盐噪声，卷积核的大小也是个奇数。

边缘检测：边缘区域像素值发生“跳跃”，一阶导数在边缘处为极值即**Sobel 算子**原理；二阶导数在边缘处为 0 即**Laplace 算子**原理。Laplace 先用 Sobel 算二阶 x/y 导数再求和。`cv2.Laplacian(src, ddepth, ksize)`。

霍夫直线变换应用举例：读取图片 → 彩色图像转灰度图像 `cv2.COLOR_BGR2GRAY` → 高斯去噪 `cv2.GaussianBlur` 可选 → 阈值处理 `cv2.threshold(imggray, 80, 255, cv2.THRESH_BINARY_INV)` → `cv2.HoughLinesP` 检测直线 → 在原图上画出检测到的直线。完整图像处理流程：读图 → 颜色空间转换 → 通道分离 → 直方图处理 → 阈值二值化 → 轮廓/直线检测 → 目标识别。磨皮滤镜原理：低通滤波去除高频斑点。

实验思考问题：(1) 为什么天然自然图像**低频成分比高频成分多**？因为自然图像中像素之间通常具有连续性，平滑区域占大多数，而边缘和纹理等高频成分相对较少。(2) **YCrCb 颜色空间相比 RGB 有什么好处**？亮度和色度分离，可以独立处理亮度(Y)而不影响颜色信息，非常适合视频压缩和传输。(3) 如果霍夫变换检测出同一位置的多条直线如何处理？合并参数相近的直线，或者增加阈值参数，使用非极大值抑制制。

四、故障排查

实验中发现无法控制 **LED 排查故障**1. 电路是否接错，如 LED 的长短脚，按开关的引脚方向；2. 代码是否错误除逻辑外语法错误是否出错；3. 怀疑原件本身是否出现故障，用 shell 或 python 运行观察 LED 是否亮灭。如果按键盘控制或者显示器没有输出则是按线开关的模块电路出现问题，选择更换按线开关或导线连接电路，若 LED 灯可亮则是按线开关或电路线路。

I2C 通信主机未收到从机数据可能的原因是从机地址设置错误 (包括 7 位地址与 8 位地址混淆、地址位计算错误或器件未初始接入设备配置不当)，其次是 SDA/ACL 接线接触或焊接、缺少上拉电阻导致总线无法被拉高，以及从机未正常工作或处于复位/本地初始化状态；此外，若之前通信异常导致从机内部状态机死锁 (SDA 被持续拉低)，或主机发送的寄存器地址超出从机有效范围。

如果发现两个轮子转动差异很大，如何确定原因？1. 观察实验现象：若实际与测量严重不符，优先怀疑霍尔编码器漏计、接线松动或 GPIO 输入异常；用示波器检测驱动板输出方波，确认 GPIO 接口状态；2. 交换编码器信号线；若“慢侧”随信号线改变，问题在编码器/接线/GPIO 计数；若慢侧仍是同一物理线，则非测量问题；若慢侧随通道改变，则驱动板驱动能力不一致；若仍是同一物理线，则电机可能损坏；4. 检查机械摩擦：多次交换后仍偏慢，检查齿轮、齿轮板、轮胎与底盘间是否存在额外摩擦。

轮子不转：(1) 电机本体——直接接电池测试是否运转；(2) 电源接法——电池电压充足、驱动器正负极接反、电机与驱动板输出未接、控制线无接动；(3) PWM 与使能——确认 PWM 引脚有输出、频率和占空比合理；(4) 驱动与逻辑——确认 L298N 逻辑供电、SV 与**精液派**模块，测试驱动板输出电压；(5) 代码配置——GPIO 引脚定义与实际接线一致，PWM 正确初始化。

一个轮子转另一个不转：(1) 锁定通道——确认不转轮对应驱动通道的使能/控制引脚接线正确；(2) 使能与 PWM——确保故障通道使能启用，示波器检测 PWM 信号，或将不转轮通道 PWM 临时接故障通道测试；(3) 驱动输出——用万表测故障通道输出电压，无输出则驱动芯片或供电异常；(4) 替换测试——故障电机接正常通道，仍不转则电机损坏；正常则原通道或代码问题；(5) 代码与接地——检查 GPIO 引脚定义与占空比，确认树莓派与驱动板匹配。

霍尔测试始终为 0：查编码器接线/供电/电压输出/GPIO 中断/上拉/bouncetime。左轮无反馈：查信号线/供电/GPIO/硬件 → 万用表 → 示波器 → 交换信号线 → 加 print 验证。

PID 占空比始终 100%：误差过大 → Kp 过大或目标不合理/传感器反馈值/PWM 幅值缺失。

差速越来越大：积分饱和 + 限幅/机械不对称/Ki 不当。

如果发生小车歪扭，**如何确定问题和优化**？1. 推机械检查：检查轮胎打滑、轮子歪斜、车轴卡滞、底盘摩擦。2. 开环测试：关闭 PID，给左右轮相同 PWM，观察运行观察速度是否接近；若差异调整初始占空比或检查编码器接线；3. 编码器检查：内网转速接近但程序差异大，重点查 B2/B1 和 GPIO 接线；4. 初始校准：比例启动困难则适当提高该侧初始占空比，使 PID 接管前两侧均趋稳。5. 顺序调参：先调 Kp 修正偏移；若左右摇摆则增大 Kd 抑振；稳定后仍偏侧则加 Ki 消稳态误差。每次只改一个参数。6. 曲线测试：长期达不到目标值检查 PWM 上限/目标过高/负载过大；曲线波动则减小 Kp 或增大 Kd；稳定但固定差值则加积分或调整目标/初始占空比。

十一、网络接口

网络分层 OSI 七层与 TCP/IP 四层对应关系：应用层、表示层、会话层对应 TCP/IP 应用层；传输层对应传输层 (TCP/UDP)；网络层对应网络层 (IP)；数据链路层和物理层对应网络接口层。典型设备：终端设备工作在最上层；网关、路由器在网络层；交换机在数据链路层；光纤、双绞线、集线器属物理层。

物理地址 48 位 (6 字节)，用冒号分隔的 6 个两位十六进制数表示，是设备出厂时给定的物理地址，世界唯一且不可变，用于链路层地址寻址。**IP 地址** 32 位 (4 字节)，用点分隔的 4 个 0-255 十进制数表示，是基于网络拓朴分配的逻辑地址，网域内唯一且可变，用于网络层地址最终目标寻路。**端口号** 0-65535，用于传输层标识特定应用程序或服务。

TCP 与 UDP TCP 基于连接，通信前三次握手，断开时四次握手，采用流模式，包头最小 20 字节，程序结构相对复杂 (需 listen、accept 等)，速度相对慢，但有应答和重发机制，保证可靠交付和数据顺序。**UDP** 无连接，采用数据报模式，包头仅 8 字节，程序结构简单，速度更快，但不保证可靠交付和数据顺序。

Socket 编程流程 (Python) 服务端：创建 socket (socket.socket(AF_INET, SOCK_STREAM)) → 绑定 IP 和端口 (bind) → 监听连接请求 (listen) → 接受客户端连接 (accept, 阻塞等待) → 收发数据 (send/recv) → 关闭连接 (close)。客户端：创建 socket → 向服务端发起连接请求 (connect) → 建立连接后收发数据 → 关闭连接。

实验一：TCP 字符回显 服务端先运行，客户端向服务端发送任意字符，服务端收到后回显相同内容。发送“exit”结束会话。若 PC 作服务端则连接失败，检查 Windows 防火墙设置，允许 pythonw 通过或新建入站规则开放指定端口。

实验二：视频流网络直播 树莓派连接 USB 摄像头，安装 motion (sudo apt-get install motion)，修改配置文件 /etc/motion/motion.conf。关键参数 stream_port 8081 (视频流输出端口)、webcontrol_port 8080 (浏览器播放端口)、stream_location on、webcontrol_location on。运行 sudo motion 后，在浏览器输入 http://树莓派 IP:8080 观看，或在 VLC 中打开网络流地址 http://树莓派 IP:8081 播放。VLC 默认缓冲 1000ms，可在“显示更多选项”中将缓冲时间设为 0 以降低延迟。

常见服务器端口 HTTP 使用 80 端口，HTTPS 使用 443 端口，FTP 控制端口使用 21 端口，SMTP 邮件发送使用 25 端口，POP3 邮件接收使用 110 端口，SSH 安全远程登录使用 22 端口，RDP Windows 远程桌面使用 3389 端口。

测试 IP 可达 使用 ping IP 地址测试网络连通性。使用 traceroute IP 地址 (Linux) 或 tracert IP 地址 (Windows) 追踪数据包路由路径。

十一、概念对比与参数调查

软件 PWM	硬件 PWM
CPU 翻转定时不占 CPU	硬件 PWM 翻转定时不占 CPU
定时查询简单 CPU 占用多	CPU 翻转定时不占 CPU
开环控制	闭环控制
无反馈无自我调节	有反馈偏差自动调
图像级灰度值操作	像素级灰度值操作
同步串行	异步串行
高时钟频率 CRC 效率高	低时钟频率抗容错
均值滤波	中值滤波
核等邻域平均平滑	核高斯加权模糊

十二、重要代码

```
1.GPIO 轮询 (按键控制 LED 真值 Q1-Q3);
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
led, bt = 21, 20
GPIO.setup(led, GPIO.OUT)
GPIO.setup(bt, GPIO.IN, pull_up_down=GPIO.PUD_UP)
ledStatus = False
bt_released = True # 标记按键是否已释放，防重复触发
try:
    while True:
        if bt_released and GPIO.input(bt) == 0: # 检测按下
            sleep(0.03) # 30ms 软件去抖
            if GPIO.input(bt) == 0: # 确认仍按下
                ledStatus = not ledStatus # 翻转 LED 状态
                GPIO.output(led, GPIO.HIGH if ledStatus else GPIO.LOW)
                bt_released = False
            elif (not bt_released) and GPIO.input(bt) == 1:
                bt_released = True # 检测松开
                sleep(0.01) # 10ms 检测间隔
            except KeyboardInterrupt:
                GPIO.cleanup() # Ctrl+C 退出前清理
3.I2C 中断+去抖
GPIO.setmode(GPIO.BCM)
GPIO.setup(led, GPIO.OUT)
GPIO.setup(bt, GPIO.IN, pull_up_down=GPIO.PUD_UP)
ledStatus = False
def my_callback(channel): # 中断回调函数
    global ledStatus
    ledStatus = not ledStatus # 翻转状态
    GPIO.output(led, GPIO.HIGH if ledStatus else GPIO.LOW)
GPIO.add_event_detect(bt, GPIO.FALLING,
    callback=my_callback, bouncetime=200) # 200ms 消抖
while True: sleep(1) # 主线程等待; Ctrl+C 退出
except KeyboardInterrupt: GPIO.cleanup()
```

```
3.I2C KS103 读写 (真值 vsbus)
bus = smbus.SMBus(1) # 初始化 I2C 总线 1
addr = 0x74 # KS103 设备地址
bus.write_byte_data(addr, 0x02, 0x00) # 向寄存器 2 写测距命令
time.sleep(0.033) # 等待 ~0.33ms 确保探测完成
high = bus.read_byte_data(addr, 0x02) # 读高 8 位 (寄存器 2)
low = bus.read_byte_data(addr, 0x03) # 读低 8 位 (寄存器 3)
dist = (high << 8) | low # 合并为 16 位距离 (mm)
4.ADC0832 读取 (真值 Q4) + 逐次逼近原理:
CS, DI, DO = 18, 17, 22 # BCM 引脚定义
GPIO.setmode(GPIO.BCM)
GPIO.setup([CLK, CS, DI], GPIO.OUT)
GPIO.setup(DO, GPIO.IN)
def read_adc(channel): # channel: 0 或 1
    GPIO.output(CS, GPIO.LOW) # 拉低片选使能芯片
    GPIO.output(DI, GPIO.HIGH) # 起始位 (Start Bit)=1
    for i in range(8): # 逐位读取 8 位数据 (MSB 先)
        GPIO.output(CLK, GPIO.HIGH) # 数据输入 (CLK, GPIO.LOW)
        GPIO.output(DI, channel) # ODD/SIGN: 0=CH0, 1=CH1
        GPIO.output(CLK, GPIO.HIGH); GPIO.output(CLK, GPIO.LOW)
        data = (data << 1) | GPIO.input(DO) # 下降沿读 DO
    GPIO.output(CS, GPIO.HIGH) # 拉片选结束通信
    return data & 0-255
Vin = val * 3.3 / 256 # 数字量换算模拟电压
```

```
5.PWM 电机控制 (软件 RPLGPIO+硬件 pigpio):
=== 软件 PWM (RPi.GPIO) ===
EA, I2, I1 = 13, 19, 26 # 左路: 使能 + 两个方向
EB, I4, I3 = 16, 20, 21 # 右路: 使能 + 两个方向
FREQ = 50 # 50Hz (不抖动, 软件 PWM 能胜任)
GPIO.setup(EA, I2, I1, EB, I4, I3), GPIO.OUT
GPIO.output(EA, I2, EB, I3), GPIO.LOW
GPIO.output(I1, I4), GPIO.HIGH # 正转: I1=H, I2=L; I4=H, I3=L
pwma = GPIO.PWM(EA, FREQ)
pwmb = GPIO.PWM(EB, FREQ)
pwma.start(20); pwmb.start(20) # 初始占空比 20%
pwma.ChangeDutyCycle(50) # 运行时改占空比: 0-100
# 结束: pwma.stop(); pwmb.stop(); GPIO.cleanup()
=== 硬件 PWM (pigpio) 更精确 ===
import pigpio
pi = pigpio.pi() # 连接 pigpio 守护进程
pi.set_PWM_frequency(13, 8000) # 18 种可选频率之一
pi.set_PWM_range(13, 100) # 占空比范围 0-100
pi.set_PWM_dutycycle(13, 30) # 占空比 30%; 设 0=关闭
# 结束: pi.set_PWM_dutycycle(13,0); pi.stop()
L298N 真值表 (EN=1 时): IN1=H/IN2=L+ 正转; IN1=L/IN2=H+ 反转;
# IN1=IN2= 刹车; EN=0= 惰行。严防电机堵转!
```

6. 霍尔编码器多线测速 (真值参考完整流程):
EA, I2, I1, EB, I4, I3, LS, RS = (13, 19, 26, 16, 20, 21, 6, 12)
GPIO.setmode(GPIO.BCM)
GPIO.setup([EA, I2, I1, EB, I4, I3], GPIO.OUT)
GPIO.setup([LS, RS], GPIO.IN)
GPIO.output([EA, I2, EB, I3], GPIO.LOW)
GPIO.output([I1, I4], GPIO.HIGH)
pwma = GPIO.PWM(EA, 50); pwmb = GPIO.PWM(EB, 50)
pwma.start(0); pwmb.start(0)
lc, rc, ls, rs = 0, 0, 0, 0 # 脉冲计数 + 速度

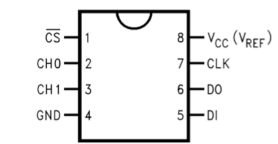
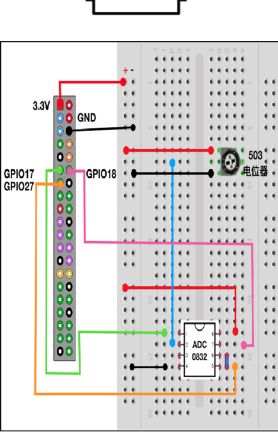
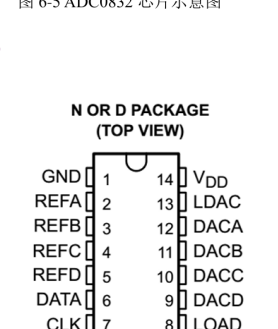


图 6-5 ADC0832 芯片示意图



(b)

```
def my_cb(ch): # 统一回调: ch 判左右
    global lc, rc
    if ch == LS: lc += 1
    elif ch == RS: rc += 1
GPIO.add_event_detect(LS, GPIO.RISING, my_cb)
GPIO.add_event_detect(RS, GPIO.RISING, my_cb)
def get_speed(): # 测速函数: 每秒计算圈/秒
    global lc, rc, ls, rs
    while True:
        time.sleep(1)
        ls = lc / 5885.0 # 588 脉冲/圈
        rs = rc / 5885.0
        lc = 0; rc = 0; # 清零累计计数
t = threading.Thread(target=get_speed)
t.setDaemon(True); t.start() # 守护线程: 主线程退出即停
# 主类: 每隔 3 秒占空比 5%, 记录速度, 绘制速度-占空比图
7.PID 类 + 双轮行军控制 (位置式+增量式双版本):
# === 位置式 PID(pid_control.py) - 完整类定义 ===
class PID:
    def __init__(self, Kp, Ki, Kd, target):
        self.Kp, self.Ki, self.Kd = Kp, Ki, Kd
        self.target = target
        self.err_pre = 0 # 当前偏差 e[k]
        self.err_last = 0 # 上次偏差 e[k-1]
        self.integral = 0 # 累积分 (需限幅防饱和)
        def update(self, feedback):
            self.err_pre = self.target - feedback
            self.integral += self.err_pre
            self.u = self.Kp*self.err_pre + self.Ki*self.integra
            self.Kd*(self.err_pre - self.err_last)
            self.err_last = self.err_pre
        if self.u > 100: self.u = 100 # 占空比上限 100%
        elif self.u < 0: self.u = 0 # 占空比下限 0%
        return self.u
# === 增量式 PID(inc_pid.py) - 核心公式 ===
du = Kp*(e[k]-e[k-1]) + Ki*e[k] + Kd*(e[k]-2*e[k-1]+e[k-2])
# du = (u[k]-u[k-1]) / dt (增量量) + 自然限幅; 不含积分累加防饱和)
I_pid = PID(Kp=30, Ki=0.05, Kd=20, target=1.5)
R_pid = PID(Kp=40, Ki=0.01, Kd=23, target=1.9)
while True:
    pwma.ChangeDutyCycle(I_pid.update(ls)) # 左轮调节
    pwmb.ChangeDutyCycle(R_pid.update(rs)) # 右轮调节
    time.sleep(0.1) # 采样间隔 0.1s
8.OpenCV 拍照+录像 (无完整程序, 真值参考):
# === 拍照 (camera_picture.py) ===
cap = cv2.VideoCapture(0) # 打开默认 USB 摄像头
while True:
    ret, frame = cap.read() # 读一帧 (ret=成功标志)
    cv2.imshow('display', frame) # 显示窗口
    key = cv2.waitKey(1) & 0xFF # 按 F 检测按键 (1ms 超时)
    if key == ord('q'): # 按 p 保存图片
        cv2.imwrite('photo.jpg', frame)
    if key == ord('q'): break # 按 q 退出
cap.release()
cv2.destroyAllWindows()
# === 录像 (camera_movie.py) ===
out = cv2.VideoWriter('movie.avi',
    cv2.VideoWriter_fourcc('X','V','I','D'),
    17, (640,480)) # 帧率 17fps, 分辨率 640x480
while True:
    ret, frame = cap.read(); out.write(frame)
    cv2.imshow('frame', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release(); out.release(); cv2.destroyAllWindows()
9.FFT 频谱 + 频域滤波: 直方图均衡化 + 阈值 >=3 平滑;
# === FFT 频谱 (fft.py) ===
y_fft = np.fft.fft(y) # FFT: y= 频域 (256 点 2^8 最佳)
fft_size = len(y)
freq = np.arange(fft_size) * fs / fft_size # 每点真实频率
mag = 2 * np.log10(np.abs(y / fft_size)) # 幅度谱 (dB); y_fft[0]=DC 分量
# === 低通滤波 500Hz(smoothing.py) ===
y_fft[[freq > 500] & (freq < fs-500)] = 0 # 高频噪声 (保留对称)
y_smooth = np.fft.ifft(y_fft.real * 1jFFT 恢复时域信号
# === 2D DFT 图像频域 ===
f = np.fft.fft2(img) # 二维 FFT
f_shift = np.fft.fftshift(f) # 零频移到中心
mag2 = 20 * np.log10(np.abs(f_shift)) # 2D 幅度谱 (中心 =DC 即平均灰度)
# === 直方图均衡化 ===
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
equ = cv2.equalizeHist(gray) # 拉伸分布增强对比度
# === 阈值二值化 ===
ret, binary = cv2.threshold(gray, 80, 255, cv2.THRESH_BINARY)
# === 霍夫直线检测 (完整流程) ===
imggray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
imggray = cv2.GaussianBlur(imggray, (3,3), 0) # 高斯去噪
ret, bw = cv2.threshold(imggray, 80, 255, cv2.THRESH_BINARY_INV)
lines = cv2.HoughLinesP(bw, 1, np.pi/180, 600, 600, 200)
# === 3x3 均值平滑滤波 (真值 Q15) ===
h, w = gray.shape
dst = np.zeros((h,w), dtype=np.uint8)
for y in range(1, h-1): # 跳过边界像素
    for x in range(1, w-1):
        s = 0
        for dy in [-1,0,1]:
            for dx in [-1,0,1]: s += gray[y+dy][x+dx]
        dst[y][x] = s // 9 # 9 邻域取平均
# === TCP 服务器端 (回显/echo) ===
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('0.0.0.0', 8888)) # 绑定本机所有 IP: 端口
server.listen(1) # 最大 1 个等待连接
conn, addr = server.accept() # 阻塞等待客户端连接
data = conn.recv(1024) # 接收最多 1024 字节
conn.send(data) # 回显 (echo) 相同内容 # UDP: SOCK_DGRAM, 无需 listen/accept/connect, 用 sendto/recvfrom
conn.close(); server.close() # TCP vs UDP: TCP 面向连接三次握手可靠有序; UDP 无连接不可靠
# === TCP 客户端 ===
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('192.168.1.100', 8888)) # 连接服务端 IP: 端口
client.send('hello') # 发送字节串
```

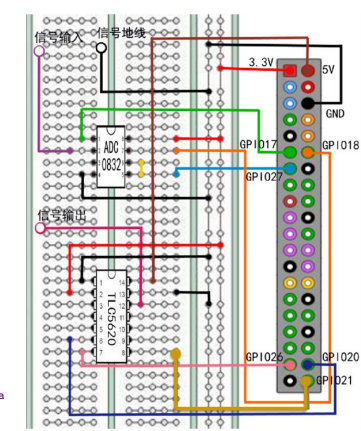


图 6-15 TLC5620 的电路连接图

Pin#	NAME	NAME	Pin#
01	3.3v DC Power	DC Power 5v	02
03	GPIO1/2 (SDA1, 1^C)	DC Power 5v	04
05	GPIO1/3 (SCL1, 1^C)	Ground	06
07	GPIO1/4 (GPIO_GCLK)	(TXD0) GPIO1/4	08
09	Ground	(RXD0) GPIO1/5	10
11	GPIO1/7 (GPIO_GEN0)	(GPIO_GEN1) GPIO1/8	12
13	GPIO1/27 (GPIO_GEN2)	Ground	14
15	GPIO1/22 (GPIO_GEN3)	(GPIO_GEN4) GPIO1/23	16
17	3.3v DC Power	(GPIO_GEN5) GPIO1/24	18
19	GPIO1/9 (SPI_MISO)	Ground	20
21	GPIO1/9 (SPI_MISO)	(GPIO_GEN6) GPIO1/25	22
23	GPIO1/11 (SPI_CLK)	(SPI_CE0_N) GPIO1/8	24
25	Ground	(SPI_CE1_N) GPIO1/7	26
27	ID_SD (I^C ID EEPROM)	(I^C ID EEPROM) ID_SD	28
29	GPIO1/5	Ground	30
31	GPIO1/6	GPIO1/12	32
33	GPIO1/3	GPIO1/26	34
35	GPIO1/19	GPIO1/16	36
37	GPIO1/26	GPIO1/20	38
39	Ground	GPIO1/21	40

```
reply = client.recv(1024) # 接收回显
client.close()
```

```
# UDP: SOCK_DGRAM, 无需 listen/accept/connect, 用 sendto/recvfrom
```

```
# TCP vs UDP: TCP 面向连接三次握手可靠有序慢; UDP 无连接不可靠快
```